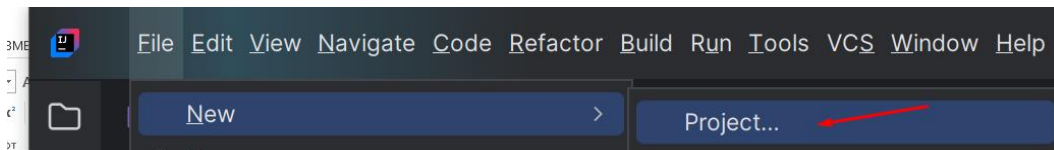


Лабораторная работа №9-10. Продвинутое ООП

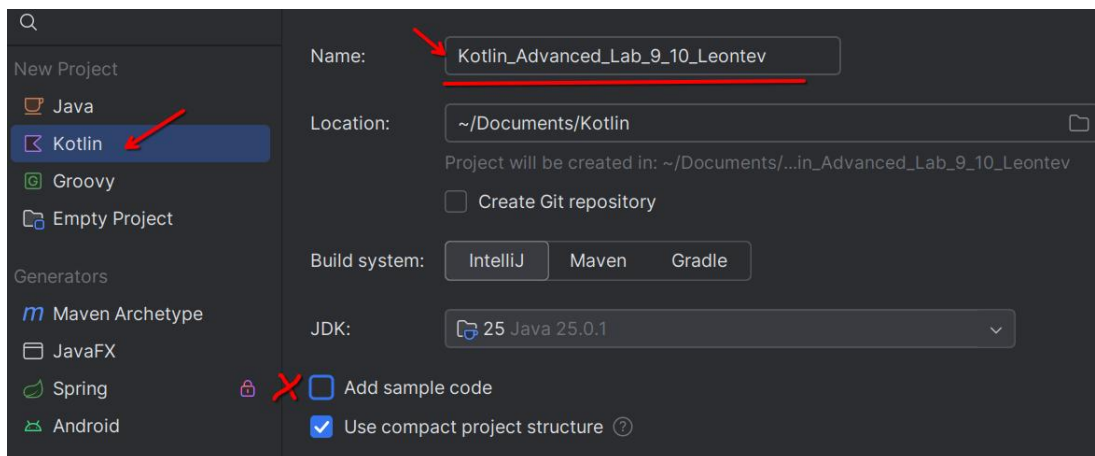
В этой серии лабораторных работ по Kotlin мы углубимся в продвинутые темы объектно-ориентированного программирования (ООП). Вы изучите ключевые концепции, такие как инкапсуляция, абстрактные классы, интерфейсы, а также дополнительные возможности **Kotlin** — **геттеры и сеттеры, абстрактные классы, data class, sealed class, object declaration, extension functions** и т.д.

Шаг 1. Создаём проект

1. Откройте **IntelliJ IDEA**.
2. Создайте новый проект: **File** → **New** → **Project**.



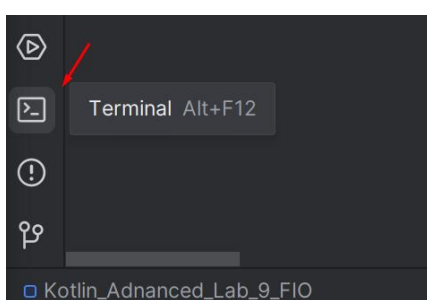
3. Выберите настройки:



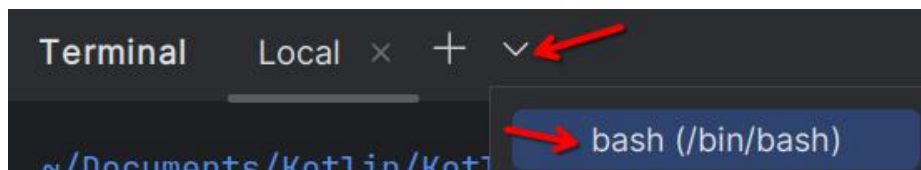
- o В качестве языка выберите **Kotlin**.
- o Укажите имя проекта: **Kotlin_Advanced_Lab_9_10_FIO** (вместо **FIO** укажите вашу фамилию).
- o Снимите галочку с **Add sample code**.
- o Нажмите **Create**.

Шаг 2. Подключаем Git

1. Откройте терминал в **IntelliJ IDEA** (**Alt+F12**).



2. Если вы привыкли к терминалу **Bash**, можете изменить его через выпадающее меню терминала.



3. Далее в примерах используется терминал **fish**, но основные команды аналогичны для **GitBash** и **PowerShell**.

Шаг 3. Инициализация Git-репозитория

1. Инициализируйте локальный репозиторий:

git init

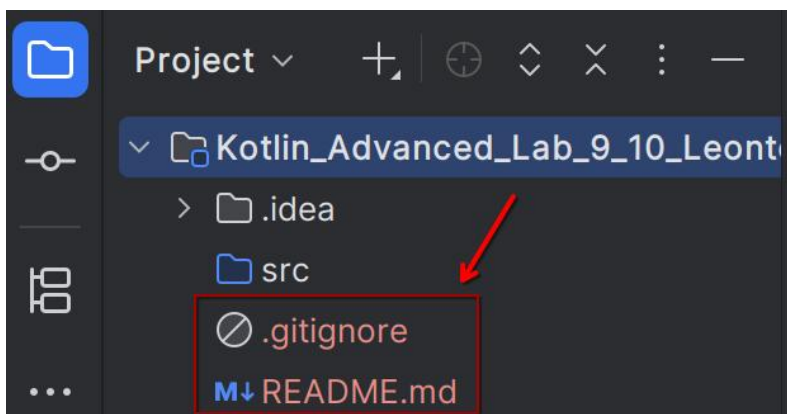
```
~/Documents/Kotlin/Kotlin_Advanced_Lab_9_10_Leontev
> git init
Инициализирован пустой репозиторий Git в
_9_10_Leontev/.git/
```

2. Создайте первый файл и проверьте его создание:

touch README.md

```
~/Documents/Kotlin/Kotlin_Advanced_Lab_9_10_Leontev main*
> touch README.md
```

Файл появится в структуре проекта вместе с автоматически созданным **.gitignore**:



В **IntelliJ IDEA** не отслеживаемые файлы (**Untracked**) отображаются красным цветом. Мы можем в этом убедиться, введя команду:

git status

```
~/Documents/Kotlin/Kotlin_Advanced_Lab_9_10_Leontev main*
> git status
Текущая ветка: main

Еще нет коммитов

Неотслеживаемые файлы:
(используйте «git add <файл>...», чтобы добавить в то, что будет включено в коммит)
.gitignore
.idea/
README.md

индекс пуст, но есть неотслеживаемые файлы
(используйте «git add», чтобы проиндексировать их)
```

Мы видим, что у нас **несколько файлов**, которые нужно добавить в индекс.

3. Добавьте файлы в индекс (staging area) и снова проверьте его статус:

git add .

git status

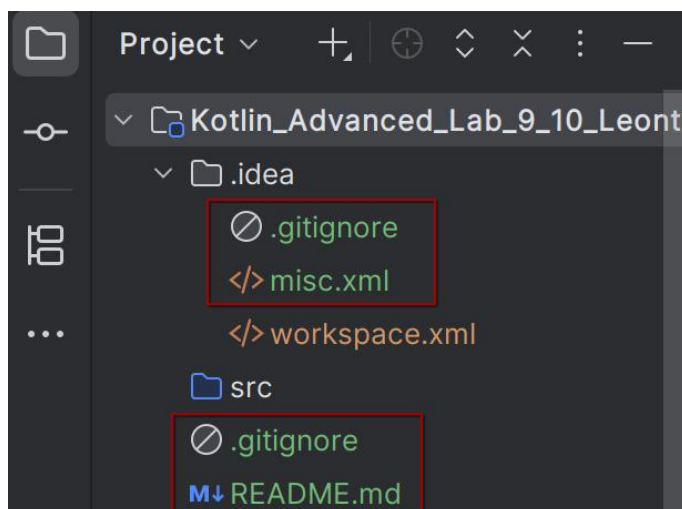
```
~/Documents/Kotlin/Kotlin_Advanced_Lab_9_10_Leontev main*
> git add .

~/Documents/Kotlin/Kotlin_Advanced_Lab_9_10_Leontev main*
> git status
Текущая ветка: main

Еще нет коммитов

Изменения, которые будут включены в коммит:
(используйте «git rm --cached <файл>...», чтобы убрать из индекса)
новый файл: .gitignore
новый файл: .idea/.gitignore
новый файл: .idea/misc.xml
новый файл: README.md
```

Теперь **файлы** отображаются **зелёным цветом** и готовы к **коммиту**. В структуре проектов они отображаются зелёным цветом:



4. Создайте коммит:

git commit -m "Initial commit"

```
~/Documents/Kotlin/Kotlin_Advanced_Lab_9_10_Leontev main*
> git commit -m "Initial commit"
[main (корневой коммит) 4fa2ee4] Initial commit
4 files changed, 99 insertions(+)
create mode 100644 .gitignore
create mode 100644 .idea/.gitignore
create mode 100644 .idea/misc.xml
create mode 100644 README.md
```

Шаг 4. Подключение репозитория GitHub

1. Подготовьте репозиторий на GitHub:

- Создайте новый пустой **public**-репозиторий:

Kotlin_Advanced_Lab_9_10_FIO (замените **FIO** на вашу фамилию).

- Без README, .gitignore и License.

The screenshot shows the GitHub 'Create repository' form. It is divided into two sections: 'General' and 'Configuration'. In the 'General' section, the 'Repository name' field is highlighted with a red box and a red arrow labeled '1'. The name 'Kotlin_Advanced_Lab_9_10_Leontev' is entered, and a green checkmark indicates it is available. In the 'Configuration' section, the 'Choose visibility' dropdown is set to 'Public' and is highlighted with a red box and a red arrow labeled '2'. The 'Add README' toggle is set to 'Off'. The 'Add .gitignore' dropdown is set to 'No .gitignore' and is highlighted with a red box and a red arrow labeled '3'. The 'Add license' dropdown is set to 'No license' and is highlighted with a red box and a red arrow labeled '4'. A red arrow points from the 'Create repository' button to the bottom of the page.

На созданной странице у вас в самом низу отобразятся нужные команды, для связывания локального репозитория с удалённым:

```
...or push an existing repository from the command line
1 git remote add origin https://github.com/Paltosik92/Kotlin_Advanced_Lab_9_10_Leontev.git
2 git branch -M main
git push -u origin main
```

ProTip! Use the URL for this page when adding GitHub as a remote.

Обратите внимание, у меня настроена по умолчанию ветка **main**, в вашем случае, скорее всего по умолчанию будет стоять ветка **master**. Проверить это можно в терминале возле названия своего репозитория или командой:

git branch

```
~/Documents/Kotlin/Kotlin_Advanced_Lab_9_10_Leontev main*  
└─ git branch  
* main
```

2. Привяжите локальный репозиторий к удалённому:

git remote add origin <URL_вашего_репозитория>

git push -u origin main/master

```
~/Documents/Kotlin/Kotlin_Advanced_Lab_9_10_Leontev main  
└─ git remote add origin https://github.com/Paltosik92/Kotlin_Advanced_Lab_9_10_Leontev.git  
  
~/Documents/Kotlin/Kotlin_Advanced_Lab_9_10_Leontev main  
└─ git push -u origin main  
Перечисление объектов: 17, готово.  
Подсчет объектов: 100% (17/17), готово.  
При сжатии изменений используется до 16 потоков  
Сжатие объектов: 100% (14/14), готово.  
Запись объектов: 100% (17/17), 2.85 КиБ | 2.85 МБ/с, готово.  
Total 17 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)  
remote: Resolving deltas: 100% (1/1), done.  
To https://github.com/Paltosik92/Kotlin_Advanced_Lab_9_10_Leontev.git  
 * [new branch]      main -> main  
branch 'main' set up to track 'origin/main'.
```

Шаг 5. Создание описания в README.md

1. Откройте файл **README.md** и добавьте в него краткое описание (более подробный разбор **Markdown** в лабораторных на дисциплине **ИСРПО**):

```
# Лабораторная работа №9-10: Продвинутое ООП на Kotlin  
  
## Описание  
Лабораторная работа посвящена изучению продвинутых концепций объектно-ориентированного программирования в Kotlin.  
  
## Структура проекта  
Проект содержит примеры реализации продвинутых механизмов ООП, а также пояснения к ключевым концепциям.  
  
## Как запустить проект  
1. Клонировать репозиторий:  
``bash  
git clone <URL_репозитория>  
``  
2. Откройте проект в IntelliJ IDEA.  
3. Запустите любой пример через контекстное меню или напрямую из `main`.  
  
## Автор  
[Ваше ФИО]  
  
## Лицензия  
Проект создан в учебных целях.
```


2. В терминале добавим файл проекта:

git add .

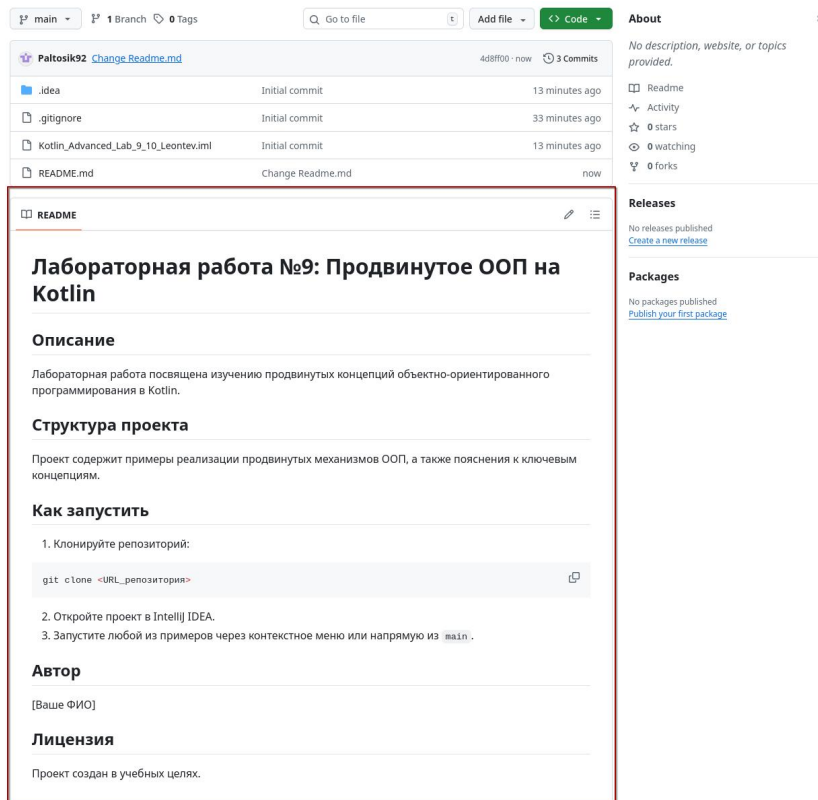
3. Создайте коммит:

git commit -m "Update README.md"

4. Запустите изменения:

git push

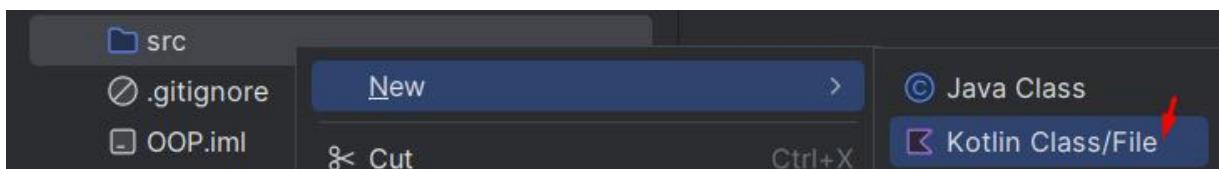
Теперь если вы перейдете в свой репозиторий на GitHub, то увидите описание:



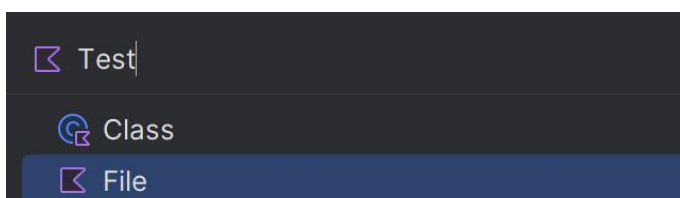
Геттеры, сеттеры и инкапсуляция

Создайте в папке **src** новый файл **Test.kt**, где мы будем писать и рассматривать базовую теорию.

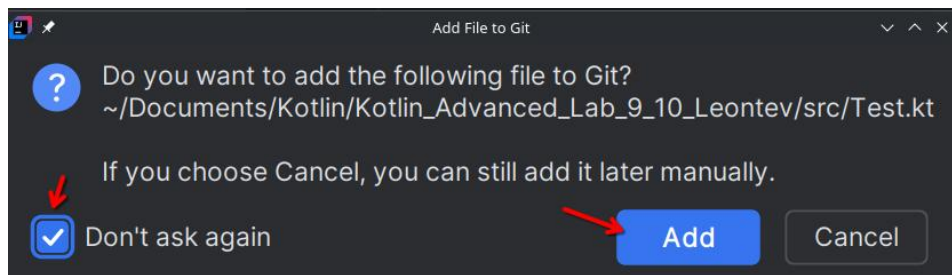
Кликните **правой кнопкой мыши** по папке **src** и выберите: **New** → **Kotlin Class/File**



Назовите новый файл **Test**:



На предложение добавить файл в **Git** соглашаемся:



Теория

Инкапсуляция — это принцип ООП, который **скрывает внутреннее состояние объекта** и позволяет управлять доступом к нему — чтобы нельзя было случайно присвоить неправильные данные.

Kotlin реализует инкапсуляцию с помощью:

- **модификаторов доступа** (private, protected, internal, public)
- **кастомных геттеров и сеттеров**
- **делегирования свойств** (lazy, observable)

Свойства в Kotlin

Свойства — это логическая комбинация **поля (field) + геттер + сеттер**.

По умолчанию у **var** есть и **getter**, и **setter**, у **val** — только **getter**.

Kotlin позволяет переопределять логику доступа. Рассмотрим пример кастомного сеттера с валидацией. Напишите следующий код:

```
var age: Int = 18 5 Usages new *
    set(value) {
        if ((value > 0) and (value < 110))
            field = value
    }

fun main() { new *
    println(age) // 18
    age = 45
    println(age) // 45
    age = -345
    println(age) // 45
}
```

Если значение не проходит проверку, поле не изменяется.

Зачем нужны кастомные геттеры/сеттеры?

- Проверка условий (валидация)
- Ограничение диапазонов

- Логирование
- Динамический пересчёт значения в getter

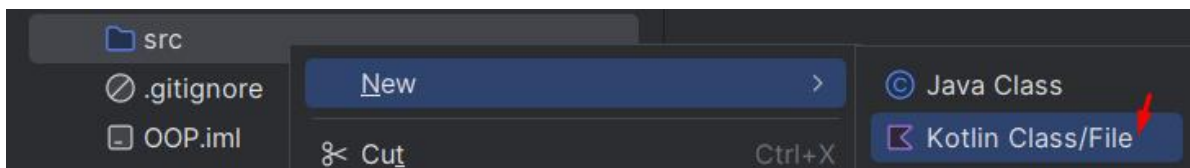
Это позволяет защитить объект от некорректного состояния.

Важный момент: field

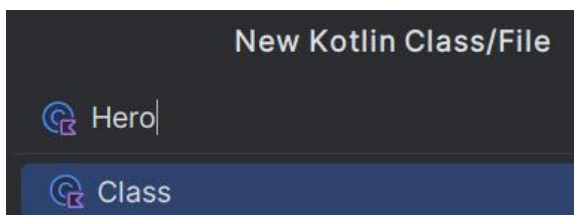
Это *скрытое* автоматически созданное хранилище данных. Внутри геттера/сеттера **только оно хранит фактическое значение** свойства.

Рассмотрим ещё один пример. Создадим отдельный класс для примера.

Кликните правой кнопкой мыши по папке **src** и выберите: **New** → **Kotlin Class/File**



Назовите новый класс **Hero**:



Шаг 6. Пример: персонаж с контролируемым состоянием

1. Для класса **Hero** установим параметр **name** — это свойство только для чтения:

```
class Hero(val name: String) {  
}
```

- **val name: String** — параметр конструктора, который автоматически становится *свойством* класса только для чтения
- в Kotlin все публичные поля автоматически имеют геттеры, а var-поля также имеют сеттеры

2. Напишем **внутри** класса свойство **health** с кастомным сеттером. Это пример сеттера с ограничением допустимого диапазона.

```
var health: Int = 100 2 Usages new *  
    set(value) {  
        field = value.coerceIn(0, 100)  
    }
```

- **field** — "закулисное" поле, в котором фактически хранится значение свойства

- **value** — параметр сеттера, содержащий присваиваемое значение
- **coerceIn(min, max)** — стандартная функция Kotlin для ограничения значения

3. Напишем внутри класса новое свойство **stamina** с кастомным геттером

```
var stamina: Int = 50
    get() = field + 10
```

- Геттер вызывается **каждый раз** при чтении свойства
- Не изменяет фактическое значение `field` (оно остаётся 50)
- Возвращает "виртуальное" значение
- Полезно для вычисляемых свойств или добавления бонусов

4. Напишем свойство **level** с приватным сеттером

```
var level: Int = 1
    private set
```

- Позволяет читать свойство отовсюду (`public get`)
- Запрещает изменять свойство извне класса
- Обеспечивает **инкапсуляцию** — сокрытие внутренней реализации

5. Напишем метод для повышения уровня

```
fun levelUp() { new *
    level++
    println("$name повысил уровень до $level!")
}
```

6. Тестирование в функции **main()**

Далее создадим за пределами класса точку входа **main()**:

```
1  > class Hero(val name: String) {...}
15
16 ▶ fun main() { new *
17
18 }
```

В ней создадим героя и установим ему здоровье:

```
val hero = Hero(name = "Утер")
hero.health = 150 // станет 100
```

Тут же выведем на консоль информацию о здоровье и выносливости:

```
println(hero.health) // 100
println(hero.stamina) // 60 (50 + 10 бонус)
```

Если мы попробуем поменять уровень, то столкнёмся с ошибкой:

```
hero.level = 10 // ошибка! private set
Cannot access 'level': it is private in '/Hero'.
Make '<set-level>' public Alt+Shift+Enter
```

Поэтому закомментируем эту строчку кода.

И далее давайте увеличим уровень героя через метод:

```
//hero.level = 10 // ошибка! private set
hero.levelUp() // увеличение уровня через метод
```

Сделаем вывод на консоль:

```
100
60
Утер повысил уровень до 2!
```

Краткий **вывод** по нашему классу:

- **health** защищён от некорректных значений
- **stamina** рассчитывается динамически
- **level** закрыт от внешнего изменения

Это и есть **инкапсуляция в действии**.

Шаг 7. Пушим изменения

1. В терминале добавим файл проекта:

git add .

2. Создайте коммит:

git commit -m "Add Test.kt and Hero class"

3. Запушьте изменения:

git push

Мини-проект, который мы будем развивать

Мы создадим компактную, но функциональную игру-симуляцию:

"Galaxy Outpost Manager"

Ты — управляющий космической базы. У тебя есть:

- **Рабочие** (Workers)
- **Ресурсы** (Resource)
- **Здания** (OutpostModule)

И мы будем постепенно усиливать систему:

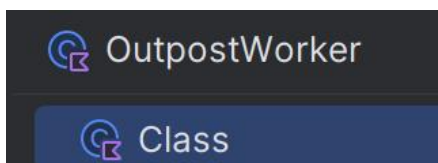
- свойства с геттерами/сеттерами
- data-классы для ресурсов
- абстрактные классы для модулей
- интерфейсы для действий и т.д.

К концу лабораторных у нас будет полноценный **мини-симулятор** с **сохранением на диск**.

Шаг 8. Начнём делать каркас проекта

Создадим первую сущность — **OutpostWorker**, класс, представляющий работника космической базы с инкапсуляцией.

1. В папке **src** создайте класс **OutpostWorker**:



Для класса **OutpostWorker** установим параметр *name*

```
class OutpostWorker(val name: String) {...}
```

1. Напишем внутри класса свойство **energy** с кастомным сеттером и ограничением диапазона

```
class OutpostWorker(val name: String) {  
    var energy: Int = 100  
    new *  
    set(value) {  
        field = value.coerceIn(0, 100)  
    }  
}
```

Назначение: Отслеживает запас энергии работника от 0 до 100

Особенности:

Начальное значение: 100 (полная энергия)

Тип var: Значение можно изменять

Кастомный сеттер: Контролирует допустимый диапазон

coerceIn(0, 100): Гарантирует, что энергия всегда остаётся в пределах 0-100

- При попытке установить значение больше 100 → сохраняется 100
- При попытке установить значение меньше 0 → сохраняется 0

2. Напишем внутри класса свойство mood — настроение. Это вычисляемое свойство, не изменяющее значение field напрямую.

```
var mood: Int = 50 new *  
    get() = field + (energy / 10)
```

Назначение: Представляет эмоциональное состояние работника

Особенности:

Базовое значение: 50 (нейтральное настроение)

Вычисляемый геттер: Настроение зависит от уровня энергии

Формула: текущее_настроение = базовое_настроение + (энергия / 10)

Динамическая зависимость: При изменении энергии автоматически меняется настроение

Пример вычисления:

При energy = 100 → mood = 50 + (100/10) = 60

При energy = 5 → mood = 50 + (5/10) = 50

3. Напишем внутри класса level — уровень развития

```
var level: Int = 1  
    private set
```

Назначение: Определяет уровень опыта/развития работника

Особенности:

Начальное значение: 1

Приватный сеттер (private set):

- о Запись доступна только внутри класса
- о **Инкапсуляция:** Защищает от прямого изменения уровня извне

4. Напишем внутри класса метод work() — выполнение работы

```
fun work() { new *  
    println("$name выполняет работу...")  
    energy -= 15  
    if (energy < 20) println("$name устал!")  
}
```

Логика работы:

1. Выводит сообщение о начале работы
2. Уменьшает энергию на 15 единиц
3. Проверяет критический уровень энергии (<20)
4. При низкой энергии выводит предупреждение

Важно: Изменение energy вызывает кастомный сеттер с проверкой границ

5. Напишем внутри класса метод `rest()` — отдых

```
fun rest() { new *
    println("$name отдыхает...")
    energy += 20
}
```

Логика работы:

1. Выводит сообщение об отдыхе
2. Увеличивает энергию на 20 единиц
3. Сеттер `energy` автоматически ограничивает значение до 100

Даже при многократном отдыхе энергия не превысит максимум.

6. Напишем внутри класса метод `levelUp()` — повышение уровня

```
fun levelUp() { new *
    level++
    println("$name повышает уровень: $level")
}
```

Логика работы:

1. Увеличивает уровень на 1 (доступно благодаря `private set`)
2. Выводит сообщение о новом уровне
3. Единственный допустимый способ повысить уровень извне

Посмотрим на работу нашего работника

В том же файле создадим точку входа `main()`. В ней создадим нашего работника, и выведем его начальное состояние:

```
fun main() { new *
    val worker = OutpostWorker(name = "Алексей")
    println("Энергия: ${worker.energy}")
    println("Настроение: ${worker.mood}")
}
```

Запустим, и проверим корректный вывод:

```
Энергия: 100
Настроение: 60
```

Далее пусть наш работник поработает и после посмотрим на то, как изменилось его состояние:


```
fun main() { new *
    val worker = OutpostWorker(name = "Алексей")
    println("Энергия: ${worker.energy}")
    println("Настроение: ${worker.mood}")
    worker.work()
    worker.work()
    worker.work()
    println("Энергия: ${worker.energy}")
    println("Настроение: ${worker.mood}")
}
```

Дадим нашему работнику отдохнуть:

```
worker.rest()
```

Попробуем повысить максимальную энергию и уровень:

```
worker.energy = 150
worker.level = 5
```

Попытка изменить уровень вызовет у нас ошибку, поэтому **закомментируем** эту строчку.

Далее повысим уровень работника:

```
worker.levelUp()
```

Запускаем и видим что уровень нашего работника повысился.

В нашем текущем коде у работника аванпоста есть серьёзное ограничение — его максимальная энергия всегда равна 100, независимо от уровня. Даже если работник повышает уровень, он не становится выносливее и не может больше работать! Это не очень логично и ограничивает геймплей.

Давайте сделаем систему более продвинутой! С точки зрения геймдизайна вместо фиксированного максимума в 100 единиц энергии, мы будем увеличивать этот максимум с каждым новым уровнем.

1: Добавляем свойство maxEnergy

```
5 Usages // Максимальная энергия — теперь это отдельное свойство
var maxEnergy: Int = 100
private set // Запрещаем внешнее изменение
```

Теперь это отдельное инкапсулированное свойство.

2. Изменяем свойство `energy`, делая его зависимым от `maxEnergy`

```
10 Usages //Текущая энергия зависит от максимальной
var energy: Int = maxEnergy
    set(value) {
        field = value.coerceIn(0, maxEnergy)
    } //Используем maxEnergy вместо жёсткой 100
```

3. Модифицируем метод `levelUp()`

Теперь повышение уровня влияет не только на счётчик, но и на характеристики.

```
fun levelUp() { // Увеличиваем максимальную энергию на 20 за каждый уровень
    level++
    + maxEnergy += 20 // Полностью восстанавливаем энергию
    + energy = maxEnergy // при повышении уровня
    println("$name повышает уровень: $level |
    Максимальная энергия увеличена до: $maxEnergy")
} // Добавляем вывод максимальной энергии
```

Ключевые концепции ООП в данном примере

Концепция	Реализация	Назначение
Инкапсуляция	private set для level	Скрытие реализации, защита данных
Кастомные аксессоры	Геттер mood, сеттер energy	Контроль доступа к свойствам
Валидация данных	coerceIn() в сеттере	Гарантия корректности состояния
Вычисляемые свойства	mood зависит от energy	Динамические значения
Поведение объекта	Методы work(), rest(), levelUp()	Определение возможностей объекта

Шаг 9. Пушим изменения

1. В терминале добавим файл проекта:

git add .

2. Создайте коммит:

git commit -m "Add OutpostWorker with getters/setters"

3. Запушьте изменения:

git push

Data-классы (data class)

Теория

Data-класс (класс данных) — это специальный тип класса в Kotlin, предназначенный в первую очередь для хранения данных. Основное преимущество data-классов — **автоматическая генерация** стандартных методов, которые обычно приходится писать вручную.

Автоматически генерируемые методы:

toString() — возвращает строковое представление объекта;

equals() — сравнивает два объекта на равенство;

hashCode() — возвращает хеш-код объекта;

copy() — копирует данные объекта в другой объект;

componentN() — функции для декомпозиции.

Используется для моделей, сущностей, записей, ресурсов, настроек.

Основные характеристики и ограничения:

1. **Обязательный параметр:** Data-класс должен иметь хотя бы один параметр в первичном конструкторе
2. **Не обеспечивается на уровне языка, а является рекомендацией:** Свойства обычно объявляются как `val` (рекомендуется), но можно использовать `var`
3. **Наследование:** Data-классы не могут быть унаследованы от других классов (кроме от `Any`)
4. **Области применения:** DTO (Data Transfer Objects), модели данных, сущности, настройки, конфигурации

Шаг 10. Практическое изучение Data-классов

В файле **Test.kt** создайте **data class**, который описывает вещь с **3 свойствами** — **id**, **имя** и **качество**:

```
data class Item( new *  
    val id: Int,  
    val name: String,  
    val quantity: Int  
)
```

Data-класс Item представляет игровой предмет

* **id** — Уникальный идентификатор предмета

* **name** — Название предмета

* **quantity** — Количество экземпляров

Метод **copy()** позволяет создавать

изменённую копию объекта без изменения оригинала.

В качестве примера, в методе **main()** в текущем файле **Test.kt** рассмотрим одну из функций метод **copy()**:

```
fun main() {  
    //...предыдущий код  
    val sword = Item(id = 1, name = "Sword", quantity = 1)  
    val betterSword = sword.copy(quantity = 2)  
    println(sword.toString())  
    println(betterSword.toString())  
}
```

Запустим программу. **Результат выполнения:**

```
Item(id=1, name=Sword, quantity=1)  
Item(id=1, name=Sword, quantity=2)
```

Без модификатора **data** метод **toString()** вернул бы:

```
Item@1be6f5c3
```

Малоинформативное представление объекта.

Кастомизация автоматически сгенерированных методов

Мы можем переопределить метод **toString()** (сочетание клавиш — **Ctrl+O**) для более читаемого вывода:

```
data class Item(...) {  
    2 Usages  
    override fun toString(): String {  
        + return "Id предмета: $id\nИмя: $name\nКоличество: $quantity\n"  
    }  
}
```

Декомпозиция (Destructuring Declaration)

Автоматическое разложение объекта на отдельные переменные.

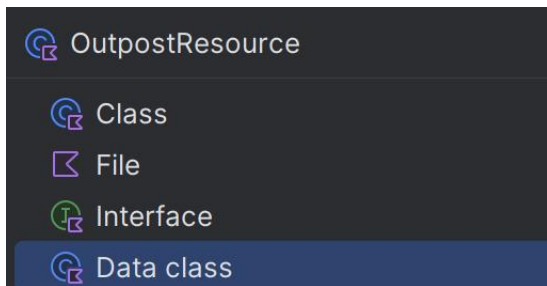
Используются сгенерированные функции **component1()**, **component2()**, **component3()**:

```
fun main() {  
    //...предыдущий код  
    ...//создание мечей  
    + [val(id, name, quantity) = betterSword  
      println("Id предмета: $id\nИмя: $name\nКоличество: $quantity\n")  
    ]
```


Шаг 11. Интеграция Data-классов в проект космической базы

Создание Data-класса OutpostResource

Создайте в папке **src** новый файл **OutpostResource.kt**:



Пропишем в нём простую структуру:

```
data class OutpostResource(  
    val id: Int,  
    val name: String,  
    var amount: Int  
)
```

id — Уникальный идентификатор ресурса

name — Название ресурса

amount — Текущее количество

Также переопределим метод **toString()**:

```
data class OutpostResource(...) {  
    override fun toString(): String { 3 Usages    new *  
        return "Ваш ресурс: $id | Имя: $name | Количество: $amount"  
    }  
}
```

В этом же классе создадим точку входа **main()**. Протестируем наш data-класс:

```
fun main() {  
    val gas = OutpostResource(id = 1, name = "Gas", amount = 100)  
    val mineral = OutpostResource(id = 2, name = "Minerals", amount = 250)  
    println("Успех! Вы добыли дополнительное количество минералов:  
        ${mineral.amount + 50}")  
    val bonusMineral = mineral.copy(id = 3, name = "Minerals Bonus", amount  
        = mineral.amount + 50)  
    println(gas.toString())  
    println(mineral.toString())  
    println(bonusMineral.toString())  
}
```

Шаг 11. Создадим менеджер ресурсов (каркас)

Создадим класс для управления ресурсами космической базы

Создайте новый класс **ResourceManager.kt**:

Мы будем хранить ресурсы в списке.

Создаём новый класс — **ResourceManager.kt**

В начале класса объявим приватный изменяемый список resources

для хранения ресурсов:

```
class ResourceManager { 1 Usage    Paltosik92
    + private val resources = mutableListOf<OutpostResource>()
```

Внутри класса создадим функцию add(), для добавления ресурса в коллекцию:

```
fun add(resource: OutpostResource) { 2 Usages    2
    resources.add(resource)
    println("Добавлен ресурс: ${resource.name}")
}
```

Внутри класса создадим функцию get() для поиска ресурса по имени:

```
fun get(name: String): OutpostResource? {  Paltosik92
    return resources.find { it.name == name }
}
```

- **find { ... }** — стандартная функция коллекций Kotlin. Она перебирает список и возвращает первый элемент, для которого условие истинно
- **it** — текущий элемент списка (OutpostResource)
- **it.name == name** — проверка совпадения имени ресурса

Внутри класса создадим функцию printAll() для вывода всех ресурсов в консоль:

```
fun printAll() { 1 Usage    Paltosik92
    println("Ресурсы базы")
    resources.forEach { println("${it.name}: ${it.amount}") }
}
```

- **forEach { ... }** — функция высшего порядка для перебора коллекции
- **it** — текущий ресурс

Метод:

- не возвращает данные

- не раскрывает коллекцию наружу
- выполняет только отображение состояния

Это пример *read-only поведения*, соответствующего принципу инкапсуляции.

Создайте файл **Main.kt** для тестирования:

📄 Main.kt

В нём создаём точку входа **main()**.

Создаём менеджер ресурсов **manager** и ресурсы **minerals** и **gas**:

```
fun main() {  👤 Paltosik92
    val manager = ResourceManager()
    val minerals = OutpostResource(id = 1, name = "Minerals", amount = 300)
    val gas = OutpostResource(id = 2, name = "Gas", amount = 100)
```

Далее добавим ресурсы в менеджер и выведем все ресурсы на консоль:

```
manager.add(minerals)
manager.add(gas)
manager.printAll()
```

Создадим копии ресурсов с бонусом и выведем его на консоль:

```
val bonus = minerals.copy(amount = minerals.amount + 50)
println("Копия минералов с бонусом: $bonus")
```

Запускаем и проверяем вывод:

```
Добавлен ресурс: Minerals
Добавлен ресурс: Gas
Ресурсы базы
Minerals: 300
Gas: 100
Копия минералов с бонусом: Ваш ресурс: 1 | Имя: Minerals | Количество: 350
```

Сохранение изменений в системе контроля версий

После завершения работы с Data-классами выполните следующие команды в терминале:

git add .

git commit -m "Add data class OutpostResource and ResourceManager"

git push

Абстрактные классы и интерфейсы

Абстрактный класс (abstract class)

Абстрактный класс — это базовый класс, от которого нельзя создать объект напрямую. Он используется для описания общей логики и структуры для группы связанных классов.

Может содержать:

- обычные методы с реализацией
- абстрактные методы (без реализации)
- свойства (абстрактные и обычные)

Используется, когда нужно:

- общее поведение и состояние
- базовый класс для иерархии объектов

Интерфейс (interface)

Интерфейс — это контракт поведения, который класс обязуется реализовать.

Особенности:

- не хранит состояние
- может содержать default-реализацию методов
- один класс может реализовывать несколько интерфейсов

Критерий	Abstract class	Interface
Наследование	только одно	несколько
Хранит состояние	Да	Нет
Назначение	Базовая логика	Поведение

Шаг 12. Практическое изучение абстрактных классов

В файле **Test.kt** создайте **abstract class Human**:

```
abstract class Human(val name: String){
    fun hello(){ new *
        println("My name is $name")
    }
}
```

```
fun main() {
    ...
    val denis: Human
    val pavel: Human = Human(name = "Pavel")
}
```

Cannot create an instance of an abstract class.

Абстрактный класс нельзя создать напрямую, но он может содержать реализацию методов.

Закомментируем строку с ошибкой.

Наследование

```
class Person(name: String): Human(name)
```

Абстрактные классы не требуют **open** — они по умолчанию предназначены для наследования.

Далее создадим объекты и вызовем у них метод **hello()**:

```
fun main() {  
    ...  
    val denis: Person = Person(name = "Denis")  
    val maksim: Human = Person(name = "Maksim")  
    denis.hello()  
    maksim.hello()  
}
```

```
abstract class Human(val name: String) {  
    1 Implementation  
    abstract var age: Int  
    2 Usages    1 Implementation  
    abstract fun hello()  
}  
  
3 Usages  
class Person(name: String) : Human(name) {  
    override var age: Int = 1  
    1 Usage  
    override fun hello() {  
        println("My name is $name")  
    }  
}
```

Абстрактные классы могут иметь абстрактные методы и свойства. Это такие функции и свойства, которые определяются с ключевым словом **abstract**. Абстрактные методы не содержат реализацию, то есть у них нет тела. А для абстрактных свойств не указывается значение. При этом абстрактные методы и свойства можно определить только в абстрактных классах.

Если класс наследуется от абстрактного класса, то он должен либо реализовать все его абстрактные методы и свойства, либо также быть абстрактным.

Так, в данном случае класс **Person** должен обязательно определить реализацию для функции **hello()** и свойства **age**. При этом, как и при переопределении обычных методов и свойств, применяется аннотация **override**.

Абстрактные свойства также можно реализовать в первичном конструкторе:

```
class Person(name: String, override var age : Int): Human(name){
    override fun hello(){ 1 Usage    new *
        println("My name is $name")
    }
}
```

Исправьте возникшие ошибки самостоятельно:

```
val denis: Person = Person(name = "Denis")
val maksim: Human = Person(name = "Maksim")
```

Зачем нужны абстрактные классы? Классы обычно отражают какие-то сущности реального мира. Но некоторые из этих сущностей представляют абстракцию, которая непосредственного воплощения не имеет. Например, возьмём систему геометрических фигур. В реальности не существует геометрической фигуры как таковой. Есть круг, прямоугольник, квадрат, но просто фигуры нет. Однако же и круг, и прямоугольник имеют что-то общее и являются фигурами. В этом случае мы можем определить абстрактный класс фигуры и затем от него унаследовать все остальные классы фигур:

```
abstract class Figure {
    1 Implementation
    abstract fun perimeter(): Float
    1 Implementation
    abstract fun area(): Float
}

class Rectangle(val width: Float, val height: Float) : Figure() {
    override fun perimeter(): Float {
        return 2 * (width + height)
    }

    override fun area(): Float {
        return width * height
    }
}
```

Абстрактный класс позволяет описать общие требования, не привязываясь к конкретной реализации.

Сохранение изменений в системе контроля версий

После завершения работы выполните следующие команды в **терминале**:

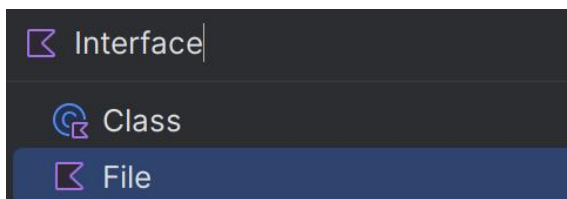
git add .

git commit -m "Add abstract class"

git push

Шаг 13. Практическое применение интерфейсов

Создадим для изучения интерфейсов новый файл **Interface.kt**:



Интерфейс представляют контракт — набор функциональности, который должен реализовать класс. Интерфейсы могут содержать объявления свойств и функций, а также могут содержать их реализацию по умолчанию. Интерфейсы позволяют реализовать в программе концепцию **полиморфизма** и решить проблему **множественного наследования**, поскольку класс может унаследовать только один класс, зато интерфейсов он может реализовать множество.

Для применения интерфейса после имени класса через двоеточие (как при наследовании) указывается имя применяемого интерфейса. Рассмотрим простой пример в файле **Interface.kt**:

```
interface Movable{}  
class Car : Movable {}
```

Пока интерфейс пустой, класс может его реализовать без обязательных методов.

Добавление метода move()

Интерфейс может определять функции без реализации. Например:

```
interface Movable{  
    fun move() // определение функции без реализации  
}
```


В данном случае интерфейс **Movable** представляет функционал транспортного средства и определяет одну функцию без реализации — функцию **move()**, которая условно предназначена для передвижения.

Чтобы у нас пропала ошибка, нужно теперь в наследуемом классе реализовать метод **move()**:

```
class Car : Movable {  
    override fun move() {  
        println("Едем на машине")  
    }  
}
```

Таким образом, у нас есть интерфейс **Movable**, который представляет абстрактное транспортное средство, и есть метод **move()**, который предназначена для перемещения транспортного средства, но как именно это перемещение осуществляется — неизвестно.

Стоит отметить, что мы не можем напрямую создать объект интерфейса, так как интерфейс не поддерживает конструкторы и просто представляет шаблон, которому класс должен соответствовать.

Создадим внутри нашего файла ещё один класс самолёта:

```
class Aircraft : Movable{  
    override fun move(){  
        println("Летим на самолете")  
    }  
}
```

Здесь определены классы **Car** и **Aircraft**, которые условно представляют машину и самолет. При применении интерфейса класс должен реализовать все его абстрактные методы и свойства. При реализации функций и свойств перед ними ставится ключевое слово **override**.

Так, класс **Car** применяет интерфейс **Movable**. Так как интерфейс содержит абстрактный метод **move()**, то класс **Car** обязательно должен его реализовать. То же самое касается класса **Aircraft**.

Создадим точку входа **main()**.

Далее мы можем вызвать реализованный метод **move()** как любую другую функцию класса:

```
fun main() {  
    val car = Car()  
    val aircraft = Aircraft()  
    car.move()  
    aircraft.move()  
}
```

Консольный вывод программы:

```
Едем на машине  
Летим на самолете
```

И реализация интерфейса также означает, что мы можем рассматривать объекты классом **Car** и **Aircraft** как объекты **Movable**.

Полиморфизм и функция **travel**

И тут в дело вступает полиморфизм. Добавим функцию **travel** внутри нашего файла:

```
17 fun travel(obj: Movable) = obj.move()
```

Функции не важно, какой именно объект передан — важно, что он реализует интерфейс **Movable**.

Обновим **main()**:

```
fun main() {  
    val car = Car()  
    val aircraft = Aircraft()  
    travel(obj = car)  
    travel(obj = aircraft)  
}
```

Это и есть полиморфизм: один интерфейс — разные реализации.

В данном случае функция **travel** (условная функция путешествия на транспорте) в качестве параметра получает объект **Movable**. Это может быть машина, и самолет, и любой другой объект, класс которого реализует интерфейс **Movable**.

Мы можем объявлять переменные типа интерфейса, но создавать объекты — только через классы, которые его реализуют.

```
fun main() {  
    val car: Movable = Car()  
    val aircraft: Movable = Aircraft()  
}
```

Множественная реализация интерфейсов

В Kotlin класс не может наследоваться от нескольких классов, зато класс может реализовать множество интерфейсов. Например, у нас есть два интерфейса:

```
interface Worker{  
    fun work()  
}  
  
interface Student{  
    fun study()  
}
```

Интерфейс **Worker** представляет работающего, а интерфейс **Student** — учащегося. А что, если нам надо определить сущность работающего студента? В этом случае мы можем реализовать в классе оба этих интерфейса:

```
class WorkingStudent(val name: String) : Worker, Student {  
    override fun work() = println("$name работает")  
    override fun study() = println("$name учится")  
}
```

И в **main()** давайте посмотрим на реализацию:

```
fun main() {  
    val pavel = WorkingStudent(name = "Pavel")  
    pavel.work()  
    pavel.study()  
}
```

Класс **WorkingStudent** реализует оба интерфейса — **Worker** и **Student**. Все реализуемые интерфейсы передаются после двоеточия через запятую.

Реализация методов по умолчанию

Интерфейс может также определять реализацию по умолчанию для своих методов. В свою очередь, класс, который реализует этот интерфейс, может принять эти методы как есть, а может и переопределить их. Например, определим в интерфейсе **Movable** функцию с реализацией по умолчанию:

```
interface Movable {  
    1 Usage    2 Implementations  
    fun move()  
    fun stop() {  
        println("Останавливаемся...")  
    }  
}
```

У самолёта переопределим метод **stop()**:

```
class Aircraft : Movable {  
    override fun move() {  
        println("Летим на самолете")  
    }  
    + override fun stop() = println("Приземляемся...")  
}
```

Внутри **main()** полетим на самолёте и приземлимся:

```
aircraft.move()  
aircraft.stop()
```

Реализация свойств

Интерфейс может определять свойства — таким свойствам в интерфейсе им не присваиваются значения. Класс же, который реализует интерфейс, также обязан реализовать эти свойства. Например, объявим **свойство скорости** для нашего интерфейса **Movable**:

```
interface Movable {  
    + var speed: Int // объявление свойства  
    fun move()  
}
```

Теперь для наших классов нужно реализовать свойства, чтобы исправить ошибку:

```
class Car : Movable {  
    0  
    Class 'Car' is not abstract and does not implement abstract member:  
    var speed: Int
```

Установим для **машины** скорость:

```
class Car : Movable {  
    + override var speed = 60
```

И изменим у него вывод в методе **move()**:

```
override fun move() {  
    ✓ println("Едем на машине со скоростью $speed км/ч")  
}
```

И для самолёта:

```
class Aircraft : Movable {  
    + override var speed = 600
```

Изменим у самолёта вывод в методе `move()`:

```
println("Летим на самолёте со скоростью $speed км/ч")
```

Запустим программу, и увидим вывод:

```
Едем на машине со скоростью 60 км/ч  
Останавливаемся...  
Летим на самолете со скоростью 600 км/ч  
Приземляемся...
```

В данном случае в интерфейсе **Movable** определено свойство **speed**. Здесь реализация свойства в классах заключается в установке для него начального значения.

Установка свойств через конструктор

Стоит отметить, что реализуемые свойства интерфейса могут устанавливаться через конструктор. Иногда это единственное место, где можно получить значения для свойств. Добавим в наш интерфейс модель и номер:

```
interface Movable {  
    var speed: Int  
    + {  
        val model: String  
        val number: String
```

Далее переопределим для машины эти свойства в первичном конструкторе:

```
class Car(  
    + (override val model: String,  
      override val number: String  
    ) : Movable {
```

И аналогично сделаем для самолёта:

```
class Aircraft (  
    + (override val model: String,  
      override val number: String  
    ) : Movable {
```


Далее в методе **main()** обновим создание объектов:

```
fun main() {  
    val car = Car(model = "LADA", number = "134LAD")  
    val aircraft = Aircraft(model = "Boeing", number = "737")  
}
```

Здесь интерфейс **Movable** также определяет свойства **model** (модель) и **number** (номер транспортного средства). Но эти характеристики различаются для каждой конкретной машины, соответственно их предпочтительнее устанавливать в конструкторе. В примере выше они устанавливаются в первичном конструкторе класса **Car**.

Правила переопределения

В Kotlin мы можем одновременно реализовать интерфейсы, которые определяют функцию с одним и тем же именем. То же самое касается ситуации, когда класс одновременно реализует интерфейс и наследует класс, которые имеют одноименную функцию. В программировании подобная проблема известна как **diamond problem** (проблема ромбовидного наследования). В этом случае класс, реализующий интерфейсы, может определить одну функцию для всех реализаций.

Создадим два интерфейса для **видео-** и **аудиопроигрывателя**:

```
interface VideoPlayable {  
    fun play()  
}  
  
interface AudioPlayable {  
    fun play()  
}
```

Далее создадим класс, который будет реализовывать оба интерфейса, и в нём переопределим метод для обоих интерфейсов:

```
class MediaPlayer : VideoPlayable, AudioPlayable {  
    override fun play() = println("Play audio and video")  
}
```

В **main()** создадим проигрыватель, и запустим у него метод воспроизведения:

```
fun main() {  
    val player = MediaPlayer()  
    player.play() // Play audio and video  
}
```

Здесь интерфейсы `VideoPlayable` и `AudioPlayable` определяют функцию `play`. В этом случае класс `MediaPlayer`, который применяет оба интерфейса, обязательно должен определить функцию с тем же именем, то есть `play`.

Вызов реализации из интерфейса

Иногда может быть необходимо использовать функцию из интерфейса с реализацией по умолчанию, но при этом добавить к ней еще какой-то функционал. В этом случае нет смысла дублировать в классе реализацию по умолчанию. И мы можем обратиться к реализации из интерфейса с помощью конструкции `super<интерфейс>.имя_функции`.

Вносим изменения в наши интерфейсы:

```
interface VideoPlayable {  
    fun play() = println("Play video")  
}
```

```
interface AudioPlayable {  
    fun play() = println("Play audio")  
}
```

Далее в классе **`MediaPlayer`** изменяем метод **`play()`**:

```
class MediaPlayer : VideoPlayable, AudioPlayable {  
    override fun play() {  
        println("Start playing")  
        super<VideoPlayable>.play() // вызываем VideoPlayable.play()  
        super<AudioPlayable>.play() // вызываем AudioPlayable.play()  
    }  
}
```

В данном случае интерфейсы **`VideoPlayable`** и **`AudioPlayable`** определяют для функции **`play()`** реализацию по умолчанию, а в классе **`MediaPlayer`** вызывается эта реализация.

Сохранение изменений в системе контроля версий

После завершения работы выполните следующие команды в терминале:

`git add .`

`git commit -m "Add interface"`

`git push`

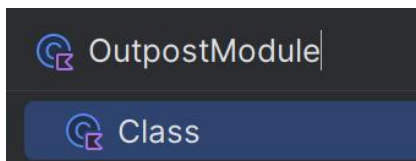
Шаг 14. Применяем абстрактные классы и интерфейсы в проекте Galaxy Outpost Manager

Нам нужен базовый модуль космической базы — **OutpostModule**

Каждый модуль должен:

- иметь имя
- иметь уровень и возможность повысить его
- уметь работать

Поэтому создаём **абстрактный класс** — **OutpostModule.kt**:



Добавим в **первичный конструктор** свойства — имя и уровень:

```
abstract class OutpostModule(  
    val name: String,  
    var level: Int = 1  
) {  
    ...  
}
```

Далее в **классе** добавим метод для повышения уровня:

```
abstract class OutpostModule(  
    ...  
) {  
    fun upgrade() {  
        level++  
        println("$name улучшен до уровня $level")  
    }  
}
```

И внутри класса добавим абстрактную функцию работы:

```
+ { abstract fun performAction(manager: ResourceManager)  
}
```

- **abstract fun performAction()** обязан быть реализован наследниками.
- **upgrade()** — общая логика для всех модулей.

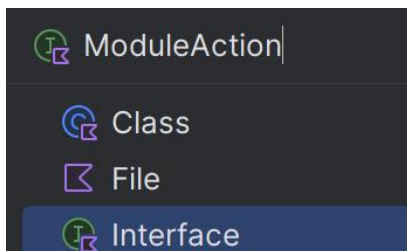
Таким образом, каждый модуль реализует свою работу, но повышается по единому правилу.

Далее делаем интерфейс действия модулей.

Каждый модуль может:

- производить ресурсы
- потреблять ресурсы
- генерировать события

Сделаем простой интерфейс — **ModuleAction.kt**, который будет определять **контракт для действий/модулей**, которые могут **взаимодействовать с менеджером ресурсов**:



Пропишем логику:

```
interface ModuleAction { new *  
    fun execute(manager: ResourceManager)  
}
```

1. **Каждый модуль/действие** должен реализовать этот интерфейс.
2. **Метод execute(manager: ResourceManager)** — это единственный обязательный метод, который получает доступ к менеджеру ресурсов для:
 - **Чтения** ресурсов (manager.get())
 - **Добавления** новых ресурсов (manager.add())
 - **Просмотра** всех ресурсов (manager.printAll())

Далее создадим наши первые модули — **два** класса **EnergyGenerator** (производит энергию) и **ResearchLab** (использует минералы для исследования):



Далее прописываем логику **EnergyGenerator**. Наследуемся от абстрактного класса **OutpostModule**:

```
class EnergyGenerator : OutpostModule(name = "Генератор энергии") {}
```

Так как класс наследуется от абстрактного класса, он обязан реализовать **performAction()**:

```
) {
    override fun performAction(manager: ResourceManager) {
    }
}
```

В начале выведем информацию что генератор работает:

```
override fun performAction(manager: ResourceManager) {
    + println("Генератор работает... Производит 20 энергии")
}
```

Далее обратимся к нашему классу **ResourceManager**, вызовем из него метод **get()** и сохраним в отдельную переменную **energy**:

```
override fun performAction(manager: ResourceManager) {
    println("Генератор работает... Производит 20 энергии")
    + val energy = manager.get("Energy")
}
```

И далее мы хотим внутри нашего метода **performAction()** реализовать два сценария:

Если энергия уже существует:

- о Увеличивает количество на 20 единиц

Если энергии нет:

- о Создает новый ресурс: OutpostResource
- о Добавляет через manager.add()

```
if (energy != null) {
    energy.amount += 20
} else {
    manager.add(OutpostResource(id = 99, name = "Energy",
        amount = 20))
}
```

Пропишем логику класса **ResearchLab**. Наследуемся от абстрактного класса **OutpostModule**:

```
class ResearchLab : OutpostModule(
    name = "Исследовательская лаборатория"
) {}
```

Класс обязан реализовать **performAction()**, так как наследуется от **OutpostModule**.


```
override fun performAction(manager: ResourceManager) {  
  
}
```

Внутри метода **performAction()** будем искать ресурсы в менеджере и сохраним в переменную **minerals**:

```
override fun performAction(manager: ResourceManager) {  
    + val minerals = manager.get("Minerals")  
}
```

Далее внутри метода **performAction()** мы осуществим проверку условий:

Если минералов нет **ИЛИ** если минералов меньше 30:

→ Вывод сообщения об ошибке и выход:

```
if (minerals == null || minerals.amount < 30) {  
    println("Недостаточно минералов для исследования")  
    return  
}
```

Если условия выполнены:

- Уменьшает количество минералов на 30 и выводим сообщение о проведении исследования:

```
minerals.amount -= 30  
println("Лаборатория проводит исследование (минералы -30)")
```

Далее в файле **Main.kt** реализуем тестовый запуск.

Закомментируйте предыдущий код.

1: Инициализация менеджера ресурсов

Создается объект **ResourceManager** с пустым списком **resource**:

```
fun main() {  
    val manager = ResourceManager()  
}
```

2: Добавление начальных ресурсов

```
manager.add(OutpostResource(id = 1, name = "Minerals", amount = 120))  
manager.add(OutpostResource(id = 2, name = "Gas", amount = 40))
```

3: Создание модулей

```
val generator = EnergyGenerator()
val lab = ResearchLab()
```

4: Запуск генератора энергии и исследовательской лаборатории

```
generator.performAction(manager)
lab.performAction(manager)
```

6: Вывод всех ресурсов

```
println()
manager.printAll()
```

Запускаем на консоль. Мы видим, как модули взаимодействуют через менеджер ресурсов, не зная о его внутренней реализации.

Сохранение изменений в системе контроля версий

После завершения работы выполните следующие команды в терминале:

git add .

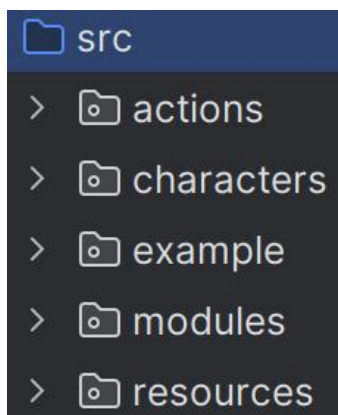
git commit -m "Add abstract modules, interfaces and first module implementations"

git push

Шаг 15. Структурирование проекта в пакеты

У нас уже создано несколько классов и файлов, которые находятся в одной папке **src**. Когда проект растёт, работать с "кучей" файлов в одной директории становится неудобно.

Решение: Мы структурируем проект, создав **пакеты (папки внутри src)** и распределим файлы по их логическому назначению.



1. Создаем папки-пакеты в src/

actions — Действия и интерфейсы

characters — Персонажи и герои

example — Примеры

modules — Модули аванпоста

resources — Ресурсы и управление ими

2. Перемещаем файлы по папкам (соглашаемся на Refactor)

В папку actions/:

- *ModuleAction.kt* (интерфейс для действий)

В папку characters/:

- *OutpostWorker.kt*
- *Hero.kt*

В папку examle/:

- *Interface.kt*
- *Test.kt*

В папку modules/:

- *OutpostModule.kt* (абстрактный класс)
- *EnergyGenerator.kt*
- *ResearchLab.kt*

В папку resources/:

- *OutpostResource.kt*
- *ResourceManager.kt*

Оставляем в корне src/:

- *Main.kt* (главный файл программы)

3. Обновляем объявления пакетов в файлах

Проходимся по нашим классам/интерфейсам, и в случае отсутствия нужных пакетов добавляем их. Например, у класса **EnergyGenerator** должны быть следующие импорты:

```
package modules

import resources.OutpostResource
import resources.ResourceManager
```

Сохранение изменений в системе контроля версий

После завершения работы выполните следующие команды в терминале:

git add .

git commit -m "Refactor: create package folders for better organization"

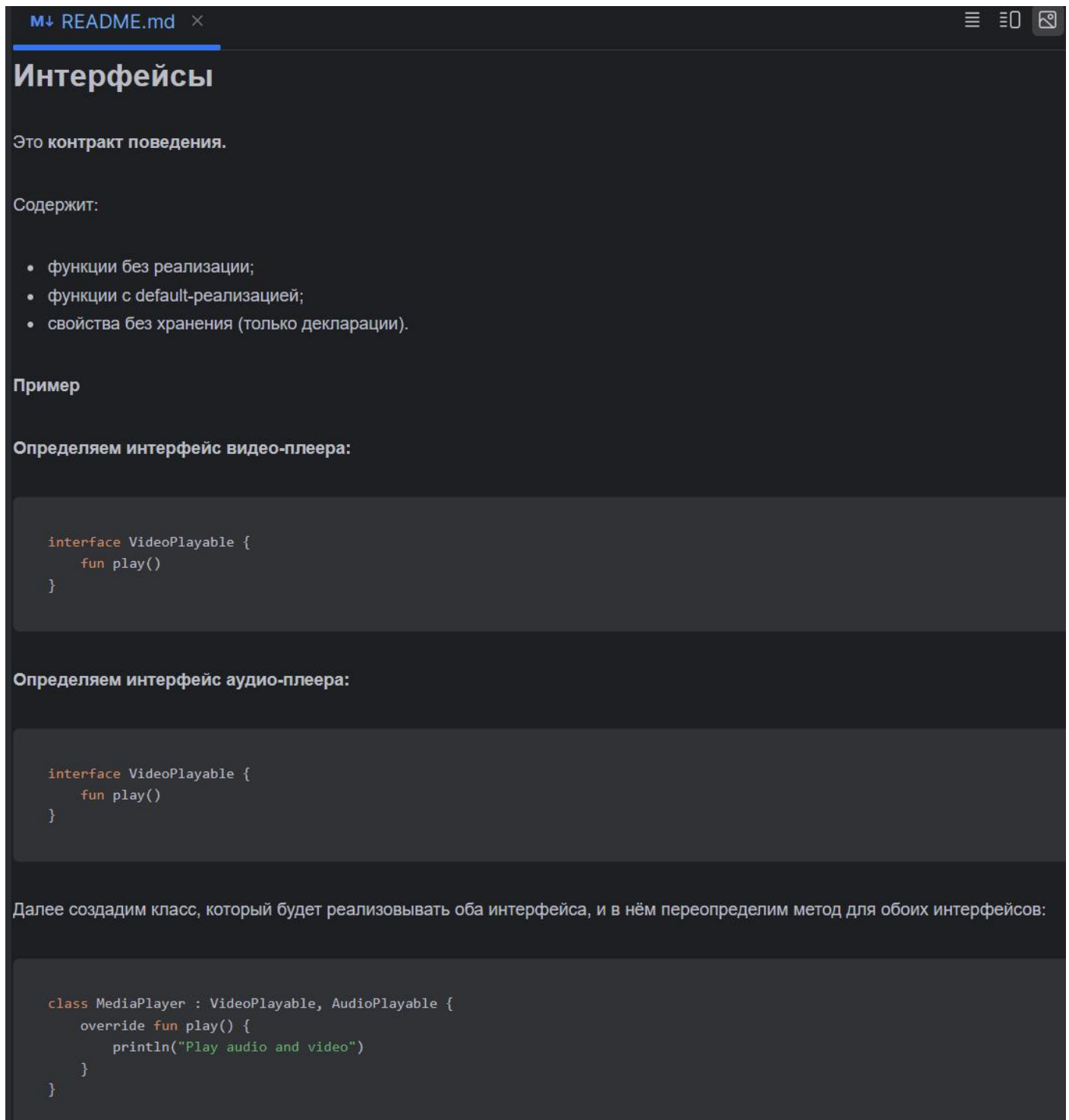
git push

Шаг 16. Добавление описания в README.md

В файле **README.md** добавьте описание и примеры тем, что мы изучили:

- **геттеры и сеттеры;**
- **инкапсуляция;**
- **data-классы;**
- **абстрактные классы;**
- **интерфейсы.**

Пример реализации для интерфейсов:



Сохранение изменений в системе контроля версий

После завершения работы с выполните следующие команды в терминале:

git add .

git commit -m "Refactor README.md"

git push

Самостоятельные задания

Задание 1: Подготовка проекта и контроль версий (Создание проекта, Git)

1. В **IntelliJ IDEA** создайте новый **Kotlin**-проект с именем **EmployeeTaskTracker**.
2. Инициализируйте в папке проекта локальный Git-репозиторий (**git init**).
3. Создайте простой файл **README.md** с названием проекта.

Выполните первый коммит (commit) с сообщением "Initial commit. Project structure"

Задание 2: Инкапсуляция, геттеры и сеттеры (Классы, свойства)

1. Создайте класс **Employee** (Сотрудник) в отдельном файле.
2. Объявите следующие **приватные (private)** свойства:
 - о **fullName** (ФИО, строка)
 - о **position** (Должность, строка)
 - о **salary** (Зарплата, целое число)
 - о **yearsOfExperience** (Опыт работы, целое число)

Выполните коммит (commit) с сообщением create "Employee class with private fields"

3. Реализуйте **публичный геттер** для всех свойств.
4. Для свойства **salary** реализуйте **сеттер**, который запрещает устанавливать отрицательное значение. При попытке установить отрицательное число, выводите в консоль предупреждение и оставляйте старое значение (или устанавливайте 0).
5. Для свойства **yearsOfExperience** реализуйте **сеттер**, который ограничивает максимальный опыт 50 годами.
6. Создайте в **main()** объект класса **Employee**, попробуйте задать некорректные значения зарплаты и опыта и убедитесь, что ваши проверки работают (вывод в консоль).

Выполните коммит (commit) с сообщением "Implement custom setters for salary and experience"

Задание 3: Data-классы

1. Создайте **data-класс Task** (Задача).
2. Добавьте свойства:
 - **title** (название),
 - **description** (описание),
 - **priority** (приоритет: **low**, **medium**, **high** — используйте **enum class**),
 - **isCompleted** (статус выполнения, по умолчанию **false**).

Выполните коммит (commit) с сообщением "Add Task data class with enum Priority"

3. В **main()** создайте 2-3 одинаковых объекта **Task** с одними данными и один отличающийся. Продемонстрируйте работу автоматически сгенерированных методов **equals()**, **hashCode()**, **copy()** и **toString()**. Например, скопируйте задачу, изменив приоритет.

Выполните коммит (commit) с сообщением "Demonstrate equals, copy and toString for Task"

Задание 4: Абстрактные классы и наследование

1. Создайте **абстрактный класс Department** (Отдел).

Выполните коммит (commit) с сообщением "Create abstract Department class"

2. Добавьте абстрактное свойство **departmentName** (название отдела) и абстрактный метод **printDepartmentGoal()** (выводит цель отдела).
3. Создайте два конкретных класса-наследника: **DevelopmentDepartment** (Разработка) и **TestingDepartment** (Тестирование).
4. Переопределите в них свойство **departmentName** (через передачу в конструктор родителя) и метод **printDepartmentGoal()**, чтобы каждый выводил свою уникальную цель (например, "Писать чистый код" и "Находить все баги").
5. В **main()** создайте объекты каждого отдела и вызовите их метод.

Выполните коммит (commit) с сообщением "Implement DevelopmentDepartment and TestingDepartment"

Задание 5: Интерфейсы

1. Создайте интерфейс **ReportGenerator**.

Выполните коммит (commit) с сообщением "Add ReportGenerator interface"

2. Добавьте в него абстрактный метод **generateReport(): String**, который возвращает текстовый отчет.
3. Реализуйте этот интерфейс в классе **Employee**. Метод должен возвращать строку с красиво оформленной информацией о сотруднике (его ФИО, должность и т.д.).
4. Реализуйте этот интерфейс в классе **DevelopmentDepartment**. Метод должен возвращать строку с информацией об отделе и его цели.
5. В **main()** создайте список (**List<ReportGenerator>**) и добавьте в него разные объекты (сотрудника и отдел). Проитерируйте по списку и выведите в консоль результаты вызова **generateReport()** для каждого элемента, демонстрируя полиморфизм.

Выполните коммит (commit) с сообщением "Demonstrate polymorphism with ReportGenerator list"

Задание 6: Интеграция*

1. Добавьте в класс **Employee** свойство **currentTask: Task?** (текущая задача, может быть **null**).
2. Добавьте метод **assignTask(newTask: Task)**, который назначает сотруднику задачу. Если у него уже есть незавершенная задача (**isCompleted == false**), выводите сообщение "Сотрудник уже занят задачей [название]".
3. В **main()** смоделируйте процесс: создайте сотрудника, отдел, несколько задач, назначьте задачу сотруднику и выведите сводный отчет.

Выполните коммит (commit) с сообщением "Implement assignTask method with validation"